MODUL 7 · DATA FETCHING & CACHING

Fetch Data di Client Components



Cara yang biasa kita pakai di React, dan sifat-sifatnya

Fetch di Client Component = Pola React

Tidak ada yang baru di sini. Ini pola React yang sudah kita kenal.

Yang dipakai

Hook React biasa

- useEffect untuk menjalankan fetch
- useState untuk menyimpan hasilnya
- File ditandai 'use client'

Yang perlu diurus

Semuanya manual

- State data, loading, dan error
- Pembatalan request
- Cache (kalau butuh)

Contoh Paling Sederhana

Ambil daftar produk dari API route, simpan di state, tampilkan.

app/produk/page.tsx

```
'use client'
import { useEffect, useState } from 'react'

export default function Page() {
  const [produk, setProduk] = useState<{ id: number; title: string }[]>([])
  useEffect(() => {
    fetch('/api/produk')
      .then((res) => res.json())
      .then((data) => setProduk(data))
  }, [])
  return <ul>{produk.map((p) => <li key={p.id}>{p.title}</li>)}</ul>
}
```

Urutan Kejadiannya

Ini penting: kodenya kelihatan sederhana, tapi urutannya tidak seperti yang dikira.

1. **Komponen render pertama kali dengan data kosong**
2. **Browser menampilkan hasil render itu**
3. **Baru setelah itu useEffect jalan dan fetch dikirim**
4. **Data tiba, setProduk dipanggil, komponen render ulang**

Contoh Menambah State Loading dan Error

Satu state untuk datanya, dua lagi untuk keadaan request.

app/produk/page.tsx (di dalam komponen)

```
type Produk = { id: number; title: string }

// data yang mau ditampilkan
const [produk, setProduk] = useState<Produk[]>([])

// true selama request masih berjalan
const [loading, setLoading] = useState(true)

// pesan kalau request gagal
const [error, setError] = useState<string | null>(null)
```

Contoh useEffect dengan try, catch, finally

fetch hanya gagal kalau jaringan putus. Status 404 atau 500 tetap dianggap berhasil, makanya kita cek res.ok.

app/produk/page.tsx (useEffect)

```
useEffect(() => {
  async function ambilData() {
    try {
      const res = await fetch('/api/produk')
      if (!res.ok) throw new Error('Gagal mengambil data')
      setProduk(await res.json())
    } catch (e) {
      setError(e instanceof Error ? e.message : 'Terjadi kesalahan')
    } finally {
      setLoading(false)
    }
  }
  ambilData()
}, [])
```

Contoh Tampilan Berdasarkan State

Tiga kemungkinan tampilan: memuat, gagal, atau datanya sudah ada.

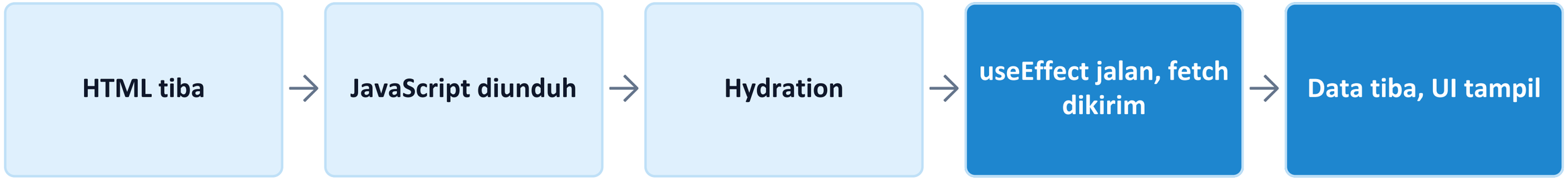
app/produk/page.tsx (return)

```
if (loading) return <p>Memuat...</p>
if (error) return <p>Error: {error}</p>

return (
  <ul>
    {produk.map((p) => (
      <li key={p.id}>{p.title}</li>
    ))}
  </ul>
)
```

Sifat 1: Baru Jalan Setelah Halaman Dimuat

Fetch menunggu semua tahap di kiri selesai dulu.



Akibatnya

Data baru diminta setelah browser selesai menyiapkan halaman. Selama itu pengguna melihat kerangka atau tulisan Memuat. Di Server Component, data sudah ikut dalam HTML yang tiba di tahap pertama.

Sifat 2: HTML Awal Belum Berisi Datanya

Ini yang kita bahas di video sebelumnya, sekarang kita lihat langsung. Hasil View Source halaman /produk.

View Source: HTML yang diterima pertama kali

```
<p>Memuat...</p>
```

Isi daftar produknya mana?

Belum ada. Kopi, Teh, dan Roti baru muncul setelah JavaScript jalan dan fetch selesai. Untuk halaman yang butuh SEO, ini kelemahannya.

Sifat 3: Request Keluar dari Browser Pengguna

Bisa dilihat siapa saja

Setiap request muncul di tab Network browser, lengkap dengan URL dan responsnya.

Kena aturan CORS

Kalau memanggil API di domain lain, server API itu harus mengizinkan domain kita.

Jangan taruh secret di sini

API key di kode client bisa dibaca siapa saja. Hanya variabel NEXT_PUBLIC_ yang sampai ke browser, dan itu pun terbaca publik.

Sifat 4 dan 5: Semua Diurus Sendiri, Tanpa Cache

Semua diurus sendiri

Berulang di tiap komponen

- State data, loading, dan error ditulis ulang
- Cek res.ok supaya status gagal terdeteksi
- Pembatalan request juga tugas kita

Tanpa cache bawaan

Diuji di Next.js 16

- Pindah halaman lalu kembali: request dikirim lagi
- Dua komponen butuh data yang sama: dua request
- Pengguna melihat Memuat lagi setiap kali

Sifat 6: Rawan Request Ganda dan Balapan

Dua hal yang diselesaikan oleh satu mekanisme: cleanup dengan AbortController.

Strict Mode (development)

Effect dijalankan dua kali

- Saat npm run dev, fetch terkirim dua kali
- Di production hanya sekali (sudah diuji)
- Request pertama sebaiknya dibatalkan

Race condition

Respons lama datang belakangan

- Pengguna pindah halaman sebelum respons tiba
- Komponen sudah hilang, state tetap diubah
- Respons lama bisa menimpa respons baru

Kode: Membatalkan Request dengan AbortController

Buat controller, kirim signal-nya ke fetch, dan batalkan di cleanup function.

app/produk/page.tsx (useEffect, bagian yang ditambah)

```
useEffect(() => {
  const controller = new AbortController()

  async function ambilData() {
    // kirim signal ke fetch
    const res = await fetch('/api/produk', { signal: controller.signal })
    ...
  }
  ambilData()

  // cleanup: jalan saat komponen hilang atau effect diulang
  return () => controller.abort()
}, [])
```

Kode: Menangani Request yang Dibatalkan

Request yang dibatalkan melempar AbortError. Itu bukan kegagalan, jadi jangan tampilkan sebagai pesan error.

app/produk/page.tsx (try, catch, finally versi final)

```
try {
  ...
} catch (e) {
  // dibatalkan itu normal, abaikan
  if (e instanceof Error && e.name !== 'AbortError') setError(e.message)
} finally {
  // jangan ubah state kalau sudah dibatalkan
  if (!controller.signal.aborted) setLoading(false)
}
```

Capek Menulis Semua Itu? Pakai Library

Next.js docs merekomendasikan SWR atau TanStack Query untuk fetch di Client Component.

Cache bersama

Data disimpan per key, jadi pindah halaman tidak selalu mulai dari nol.

Request tidak dobel

Dua komponen dengan key yang sama berbagi satu request.

Revalidate otomatis

Bisa refresh saat tab kembali aktif atau berkala (polling).

Loading dan error siap pakai

Tidak perlu tiga useState manual lagi.

Apa Itu SWR?

Library kecil berupa hook React untuk fetch data di client, dibuat oleh tim Vercel (pembuat Next.js).

Namanya

Stale-While-Revalidate

- Stale: data lama yang masih tersimpan di cache
- Revalidate: ambil data terbaru di belakang layar

Cara kerjanya

Tampil dulu, perbarui kemudian

- Tampilkan data dari cache seketika
- Fetch ulang di belakang layar
- Kalau data baru tiba, tampilan diperbarui

Hasilnya untuk pengguna

Pindah halaman lalu kembali tidak lagi mulai dari Memuat, karena data lama langsung tampil.

Kode: Contoh dengan SWR

Setelah npm install swr. Hasil yang sama, kodenya jauh lebih pendek.

app/swr/page.tsx

```
'use client'
import useSWR from 'swr'
type Produk = { id: number; title: string }

const fetcher = (url: string): Promise<Produk[]> =>
  fetch(url).then((r) => {
    if (!r.ok) throw new Error('Gagal mengambil data')
    return r.json()
  })
export default function Page() {
  const { data, error, isLoading } = useSWR('/api/produk', fetcher)
  if (isLoading) return <p>Memuat...</p>
  if (error) return <p>Error: {error.message}</p>
  return <ul>{data?.map((p) => <li key={p.id}>{p.title}</li>)}</ul>
}
```

Membaca useSWR: Key, Fetcher, dan Hasilnya

Tiga bagian dari satu baris useSWR('/api/produk', fetcher).

Key: '/api/produk'

Nama pengenal data di cache. Komponen dengan key yang sama berbagi data dan request. Biasanya berupa URL-nya.

Fetcher: fetcher

Fungsi yang kita tulis sendiri. SWR memanggilnya dengan key sebagai argumen dan memakai hasilnya sebagai data.

Hasil: data, error, isLoading

Pengganti tiga useState manual kita tadi. SWR yang mengatur isinya.

Kapan Fetch di Client Masuk Akal?

Pegangan: kalau data dibutuhkan sejak halaman pertama tampil, jangan fetch di client.

Cocok di Client

Data dibutuhkan setelah interaksi

- Pencarian dan autocomplete
- Load more dan filter dinamis
- Data yang terus diperbarui (polling)

Lebih baik di Server

Data awal halaman

- Konten utama halaman
- Halaman yang butuh SEO
- Data yang butuh secret atau database

Opsi ketiga dari docs

Ambil data di Server, oper Promise ke Client Component, lalu baca dengan use(). Kita bahas di video Suspense dan Streaming.